THREADING FOUNDATIONS

The vocabulary behind every Java concurrency interview question.

Creating work

① extend Thread

Override `run()`, then call `start()`. Simple for learning, but it couples the task to the thread.

② implement Runnable

Preferred foundation: separates what to run from where it runs. Executors accept tasks.

Thread lifecycle

NEW
created

RUNNABLE
ready / running

BLOCKED
waiting for monitor

WAITING
parked indefinitely

TIMED_WAITING
waiting with timeout

TERMINATED
finished

The concurrency problem

`race condition` — result depends on unpredictable interleaving

`critical section` — code touching shared mutable state

`mutual exclusion` — only one thread enters at a time

`thread safety` — correct under concurrent access

the rule I keep

Prefer **immutable data**, confinement, and higher-level concurrency utilities before sharing mutable state protected by locks.

⚠ interview trap

`run()` is a normal call.
`start()` creates the new execution path.

next →
intrinsic and explicit locking.

#Java #Multithreading #LearningInPublic

LOCKS & LIVENESS

Correctness is not enough: threads must also keep making progress.

Intrinsic vs explicit locks

intrinsic lock

`synchronized` uses an object's monitor. Reentrant and automatically released when the block exits.

explicit lock

`Lock` adds timed / interruptible acquisition, `tryLock()`, fairness options, and multiple conditions.

```
Lock lock = new ReentrantLock();
lock.lock();
try {
  updateSharedState();
} finally {
  lock.unlock();  // always in finally
}
```

ReentrantLock & fairness

`reentrant` — owner can acquire the same lock again

`unfair (default)` — higher throughput; barging is allowed

`tryLock()` — avoid waiting forever; can use a timeout

`fair` — longest-waiting thread favored; usually slower

Liveness failures

deadlock

threads wait forever in a lock cycle

starvation

a thread rarely gets CPU or a lock

livelock

threads react forever but make no progress

ReadWriteLock

`readLock()` allows multiple readers; `writeLock()` is exclusive. Useful only when reads dominate and operations are long enough to repay the overhead.

⚠ interview trap

Fair locks reduce starvation risk, but do not automatically prevent deadlock.

next →

visibility, atomicity, and coordination.

#Java #Locks #Concurrency

VISIBILITY & COORDINATION

Choose the smallest tool that expresses the relationship between threads.

wait / notify / notifyAll

```
synchronized (monitor) {
  while (!condition) monitor.wait();  // releases monitor
  consume();
}
synchronized (monitor) {
  produce();
  monitor.notifyAll();                // waiters compete again
}
```

*Call them only while owning the **same monitor**. Test the condition in a `while`, not an `if`, because wake-ups can be spurious or the condition can change again.*

Visibility is not atomicity

volatile

Latest write becomes visible across threads and ordering is constrained. Great for a status flag. `count++` *still races.*

Atomic*

Lock-free atomic operations such as `incrementAndGet()` *and* `compareAndSet()`*. Best for one independent value.*

One-shot vs reusable gates

`CountDownLatch` — *N events finish → waiting thread continues; one-shot*

`latch roles` — *workers count down; coordinator awaits*

`CyclicBarrier` — *N peer threads meet → all continue; reusable*

`barrier roles` — *the participating workers await each other*

happens-before

The interview idea connecting these tools: unlock → later lock, volatile write → later read, and thread start / join establish guaranteed visibility.

⚠ interview trap

`sleep()` *keeps held locks.*

`wait()` *releases the monitor.*

next →

Executors, pools, Future, CompletableFuture.

#Java #Atomic #ThreadCommunication

MODERN JAVA CONCURRENCY

Submit tasks to an execution policy—do not manually create a thread per request.

Executor framework

`Executor` — execute a Runnable

`ExecutorService` — submit, invoke, shutdown; returns Future

`ScheduledExecutorService` — delayed and periodic tasks

`Callable<T>` — returns a value and may throw checked exceptions

Thread pools

why pool?

Reuse workers, cap concurrency, queue work, control overload, and centralize lifecycle and error handling.

production rule

Size for the workload. Prefer an explicit `ThreadPoolExecutor` with a bounded queue and rejection policy.

CompletableFuture

```
CompletableFuture
  .supplyAsync(this::load, executor)
  .thenApply(this::transform)
  .thenCompose(this::saveAsync)
  .exceptionally(ex -> fallback());
```

`thenApply` maps a value. `thenCompose` flattens another async stage. Also know `allOf`, timeouts, exception handling, and the executor each stage uses.

Interview-ready next steps

`must add` — interrupts, join, Callable / Future, concurrent collections

`deeper level` — Java Memory Model, happens-before, CAS, ForkJoinPool

`modern Java` — virtual threads and structured concurrency concepts

⚠ interview trap

Always shut down an ExecutorService. A pool is a resource, not a static afterthought.

roadmap complete ✓

Learn the concept, write a demo, explain the trade-off.

#Java #ExecutorFramework #CompletableFuture